The Professor and the Waiting Line

a story to understand the event loop

DR

Dinesh Rawat

Instructor · Shorter Loop × Snapied

> Read this like a story first. Do not worry about the technical words. At the end, every part of the story maps onto how JavaScript actually works.

1 The story

Prof. Sharma walks into the lecture hall. She has one mouth and one mind, so she can only deal with one thing (one query) at a time.

She begins teaching. A student raises a hand: "Ma'am, what's the deadline?" She knows it instantly, "Friday, 5 PM", and continues. A quick thing, handled on the spot, no waiting.

Another student asks her to solve a small sum on the board. She does it herself, right there, in ten seconds. Still quick, on the spot. Even a little task she does herself is fine, as long as it is fast. Nobody waits.

Then a student, Priya, asks something slow: "Ma'am, can you check if my assignments were graded?" The records are in her office. If she stopped the whole lecture to go dig through the files, sixty students would sit frozen, waiting on her. This is the real problem.

So she doesn't. She says, "I'll check after the class. Go wait outside my office." The student leaves the classroom. That slow work is now handled somewhere else, and the professor is free to keep teaching. She never stopped.

> This is the whole trick. The slow request didn't freeze her. It got sent away to happen on its own, and she carried on serving everyone else.

She rolls on. Another quick question, by Harshit: "What will we study in the next class?" She immediately answers, "Angular." Answered, no wait needed, and she keeps teaching.

Another student asks: "Can you provide us the score cards of the previous semester?" Again: "I'll check after the class. Go wait outside my office."

Now the office has this line forming outside:

```
Priya  |  Harshit  |  Dinesh  |  Unishka  |  Gaurav  …
```

Priya does not pick up her own sheet. Suppose her sheet was found in less than a second. Priya still will not take it herself. She waits for Prof. Sharma to come, and Prof. Sharma will not come until her class is completely over.

So the line moves on, always the one who has waited longest first:

```
Harshit  |  Dinesh  |  Unishka  |  Gaurav  …
```

> The rule in her head: "As long as the class is running, don't touch the line. The moment I am free, call the next one from the line." She keeps checking this, again and again, forever. That constant checking is the loop.

2 What each part means

Prof. Sharma = the call stack. One thing at a time. (LIFO)

The line outside the office = the queue. First to arrive, first served. (FIFO)

Looking for the records inside the office = the Web / Node API. The slow work is happening here.

Records found, student now waits in line = the job moves into the queue.

The event loop = Prof. Sharma repeatedly checking "am I free now? Then call the next one." That check, over and over, is the event loop.

3 What is Node.js?

Node.js is a way to run JavaScript outside the browser, for example on a server. It uses Google's V8 engine to run the code, and adds tools to work with files, databases and the network. Node runs your JavaScript on a single thread, one thing at a time, and uses the event loop to handle slow jobs without freezing.

4 The event loop

The event loop watches two things:

- the call stack (where code runs, one thing at a time)
- the queue (where background jobs and events wait)

The rule: when the stack is empty, take the first job from the queue and run it. It checks this again and again, forever. That is the loop.

5 The four parts

- Call stack – one item at a time.
- Web / Node APIs – the background jobs: timer, file reader, database.
- Queue – the waiting line.
- Event loop – stack empty? Move the next job from the queue into the stack.